基础课

- 图的存储
  - cpp
  - python
- DFS
  - AcWing 842. 排列数字
  - AcWing 843. n-皇后问题
- BFS
  - AcWing 844. 走迷宫
  - AcWing 845. 八数码
- 树与图的深度优先遍历
  - acwing846 树的重心
    - cpp
    - python
- 树与图的广度优先遍历
  - AcWing 847. 图中点的层次
- 拓扑序列(有向图)
  - AcWing 848. 有向图的拓扑序列
- 最短路问题
  - 单源最短路
    - 所有边权都为正数
    - 存在负权边
- 多源汇点最短路
- Dijkstra
  - AcWing 849. Dijkstra求最短路 I
  - AcWing 850. Dijkstra求最短路 II
    - cpp
    - python
- bellman-ford （有边数限制）
  - AcWing 853. 有边数限制的最短路
- spfa
  - AcWing 851. spfa求最短路
  - AcWing 852. spfa判断负环
    - spfa
    - bellman-ford
- floyd
  - AcWing 854. Floyd求最短路
- 最小生成树
- Prim
  - AcWing858. Prim算法求最小生成树
  - AcWing 859. Kruskal算法求最小生成树
- 二分图
- 染色法
  - AcWing 860. 染色法判定二分图
- 匈牙利算法：
  - AcWing 861. 二分图的最大匹配

提高课

- 多源BFS
  - AcWing 173. 矩阵距离
- DFS之连通性模型
  - AcWing 1112. 迷宫

AcWing 1113. 红与黑
BFS
DFS
DFS之搜索顺序
AcWing 1116. 马走日
AcWing 1117. 单词接龙
DFS之剪枝与优化
AcWing 165. 小猫爬山
AcWing 166. 数独

基础课

图的存储

cpp

链式前向星

```cpp
int h[N], e[M], ne[M],  idx;
//idx表示第n条边
//h[N] : 表示 第 i 个节点的 第一条边的 idx
//ne[M] : 表示 与 第 idx 条边 的 下一条边 的 idx
//e[M] : 表示 第idx 条边的 终点

void add(int a,int b)
{
    e[idx] =  b;            //记录终点节点
    ne[idx] = h[a];        //相当于插入链表 ，将此节点后面的值传进来
    h[a] = idx ++;         //h[a]指向当前新增的节点
}
```

python

```python
graph = {}
n = int(input().strip())
for i in range(n-1):
    a, b = map(int, input().split())
    if a not in graph:
        graph[a] = [b]
    else:
        graph[a].append(b)
    ### 存储无向图,建双向边
    if b not in graph:
        graph[b] = [a]
    else:
        graph[b].append(a)
# 初始化状态数组
st = [False for i in range(n+1)]
```

```python

def dfs(u):
    st[u] = True

    for i in range(graph[u]):
        if not st[i]:
            dfs(i)

```

	数据结构	空间	
DFS	stack	O(n)	
BFS	queue	$O(2^n)$	最短路

算法	优点	缺点	适用问题
DFS	空间效率高，适合深层遍历	不保证最短路径，可能栈溢出	路径搜索、回溯、拓扑排序
BFS	保证最短路径，适合层次遍历	空间占用大（存储整层节点）	最短路径、扩散问题、层次分析

DFS

- 采用stack
- 回溯——恢复状态
- 剪枝——提前回溯

遇到诸如放置、字典序等可使用深搜输出全部组合。

AcWing 842. 排列数字

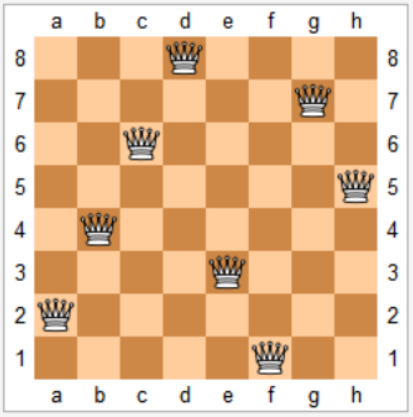
> 给定一个整数 n，将数字 $1 \sim n$ 排成一排，将会有很多种排列方法。
>
> 现在，请你按照字典序将所有的排列方法输出。
>
> **输入格式**
>
> 共一行，包含一个整数 n。
>
> **输出格式**
>
> 按字典序输出所有排列方案，每个方案占一行。
>
> **数据范围**
>
> $1 \le n \le 7$
>
> **输入样例：**
>
> ```
> 3
> ```
>
> **输出样例：**
>
> ```
> 1 2 3
> 1 3 2
> 2 1 3
> 2 3 1
> 3 1 2
> 3 2 1
> ```

```cpp
#include <iostream>

using namespace std;

const int N = 100;

int n;  //记录深度

int path[N];
int st[N];

void dfs(int u) //u控制路径长度和路径索引
{
    if (u== n)  //path形成，输出
        {
            for (int i = 0 ;i < n; i ++)
            printf("%d", path[i]);
            puts("");
            return;
        }
    for (int i = 1; i <= n; i ++)   //可以访问的子路径
        {
            if(!st[i])
                path[u] = i;    //记录路径
                st[i] = true;
                dfs(u + 1);
```

```
                st[i] = false;
        }
}


int main()
{

    cin >> n;

    dfs(0);

    return 0;
}
```

AcWing 843. n-皇后问题

n—皇后问题是指将 n 个皇后放在 $n \times n$ 的国际象棋棋盘上，使得皇后不能相互攻击到，即任意两个皇后都不能处于同一行、同一列或同一斜线上。



One solution to the eight queens puzzle

现在给定整数 n，请你输出所有的满足条件的棋子摆法。

输入格式

共一行，包含整数 n。

输出格式

每个解决方案占 n 行，每行输出一个长度为 n 的字符串，用来表示完整的棋盘状态。

其中 `.` 表示某一个位置的方格状态为空，`Q` 表示某一个位置的方格上摆着皇后。

每个方案输出完成后，输出一个空行。

注意：行末不能有多余空格。

输出方案的顺序任意，只要不重复且没有遗漏即可。

通过枚举每行（若n皇后问题能解决，则每行必有一个皇后），再通过行列表示主对角线和副对角线，判断条件符合则递归调用dfs。

- **所有主对角线上的点**，其坐标差 $i-j$ 相同。(x+1)-(y+1) = x-y
- **所有副对角线上的点**，其坐标和 $i+j$ 相同。(x-1)-(y-1) = x+y

```cpp
#include <iostream>

using namespace std;
const int N  = 200;
int n;
char g[N][N];

bool col[N], dg[N], udg[N];
//列    column
//行    line
//对角线    diagonal


void dfs(int u)
{
    if (u == n)
        {
            for (int i = 0 ;i < n; i ++)    puts(g[i]); //char* 型指针
            puts("");
            return;
        }
    for (int i = 0; i < n; i ++)       //枚举列
        {
            if(!col[i] && !dg[u + i] && !udg[n - u + i])
               {
                g[u][i] = 'Q';
                col[i] = dg[u + i] = udg[n - u + i] = true;
                dfs(u + 1);
                col[i] = dg[u + i] = udg[n - u + i] = false;
                g[u][i] =  '.';
               }
        }
}

int main()
{
    cin >> n;
    for(int i = 0; i < n; i ++)
        for(int j = 0; j < n; j ++)
            g[i][j] = '.';
    dfs(0);
    return 0;
}
```

BFS

- 采用queue
- 边权都为1时，为最短路径
- DP是特殊的最短路问题

AcWing 844. 走迷宫

给定一个 $n \times m$ 的二维整数数组，用来表示一个迷宫，数组中只包含 0 或 1，其中 0 表示可以走的路，1 表示不可通过的墙壁。

最初，有一个人位于左上角 $(1,1)$ 处，已知该人每次可以向上、下、左、右任意一个方向移动一个位置。

请问，该人从左上角移动至右下角 (n,m) 处，至少需要移动多少次。

数据保证 $(1,1)$ 处和 (n,m) 处的数字为 0，且一定至少存在一条通路。

输入格式

第一行包含两个整数 n 和 m。

接下来 n 行，每行包含 m 个整数（0 或 1），表示完整的二维数组迷宫。

输出格式

输出一个整数，表示从左上角移动至右下角的最少移动次数。

数据范围

$1 \le n, m \le 100$

BFS算法相当于在有m个后继点的点处分裂为m个小鼠，同时将访问过的点在数组上标定距离（该距离是最小值，因为其他点在相同步数下并不能访问该点），最后返回要求的坐标即可。

```
#include <iostream>
#include <utility>
#include <queue>

using namespace std;
typedef long long LL;
const int N = 1e2 +10;
int arr[N][N];
int d[N][N];
int n,m;
typedef pair<int,int> PA;
typedef pair<int,PA> PA2  ;

int xway[] = {-1,0,0,1};
int yway[] = {0,-1,1,0};


void  bfs(int x, int y)
{

    queue<PA> qu;
    qu.push({x,y});
    //qu.push({0,{x,y}});        使用d[][]来存储距离提升程序运行速率。
    while(!qu.empty())
```

```
    {
        auto tmp = qu.front();
        qu.pop();
        for(int i = 0; i < 4; i++)
        {
            int xx = tmp.first + xway[i];
            int yy = tmp.second + yway[i];
            if(xx > 0 && xx <= n && yy > 0 && yy <= m && arr[xx][yy]!= 1 && d[xx][yy]==0)
            {
                d[xx][yy] = d[tmp.first][tmp.second] + 1;
                qu.push({xx,yy});
            }
        }
    }

}


int main(){
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
            cin >> arr[i][j];
    bfs(1,1);
    cout << d[n][m] << endl;

}
```

AcWing 845. 八数码

在一个 3×3 的网格中，$1 \sim 8$ 这 8 个数字和一个 `x` 恰好不重不漏地分布在这 3×3 的网格中。

例如：

```
1 2 3
x 4 6
7 5 8
```

在游戏过程中，可以把 `x` 与其上、下、左、右四个方向之一的数字交换（如果存在）。

我们的目的是通过交换，使得网格变为如下排列（称为正确排列）：

```
1 2 3
4 5 6
7 8 x
```

例如，示例中图形就可以通过让 `x` 先后与右、下、右三个方向的数字交换成功得到正确排列。

交换过程如下：

```
1 2 3   1 2 3   1 2 3   1 2 3
x 4 6   4 x 6   4 5 6   4 5 6
7 5 8   7 5 8   7 x 8   7 8 x
```

现在，给你一个初始网格，请你求出得到正确排列至少需要进行多少次交换。

字符串上BFS这题我们从输入的状态出发，直到到达最终的状态，本质是bfs算法，通过枚举所有可能的变换，在变换的基础上再进行枚举，直到匹配最终状态。

```
记录一个错误的方向数组
int way[] = {-3,+3,-1,1};

1 2 3
x 4 6
7 5 8

若取x+way[2]，则swap(x,3)不符合题意
因此对于该题，应该先转化为x，y的
```

- 通过`<unordered_map>`存储该状态的最短路径
  - 若`.count()`存在，说明已经访问过且当前路径不是最短路径
  - 若`!.count()`，则说明未访问该路径，添加到map
  - 为什么使用`<unordered_map>`
    - <unordered_map>能保证O(1)的访问时间，但是不能进行排序
    - `<map>`访问时间为(logn)，按键自动排序

```
#include <iostream>
#include <queue>
```

```cpp
#include <string>
#include <unordered_map>
using namespace std;

string finnal = "12345678x";
unordered_map<string, int> un_map; //存放状态
queue<string> status;              //存放小鼠

int bfs(string st) {
  un_map[st] = 0;
  status.push(st);

  int xway[4] = {-1, 0, 1, 0};
  int yway[4] = {0, -1, 0, 1};

  while (status.size()) {
    string tmp = status.front();
    status.pop();
    int pos = tmp.find('x');
    int x = pos / 3;
    int y = pos % 3;
    int distance = un_map[tmp];
    if (tmp == finnal)
      return distance;

    for (int i = 0; i < 4; i++) {
      int a = x + xway[i], b = y + yway[i];     //行列移动转化为字符串加减
      if (a >= 0 && a < 3 && b >= 0 && b < 3) {
        swap(tmp[pos], tmp[3 * a + b]);
        if (!un_map.count(tmp)) {
          un_map[tmp] = distance + 1;
          status.push(tmp);
        }
        swap(tmp[pos], tmp[3 * a + b]); //恢复现场
      }
    }
  }
  return -1;
}

int main() {
  string st;
  for (int i = 0; i < 9; i++) {
    char tmp;
    cin >> tmp;
    st += tmp;
  }
  cout << bfs(st) << endl;
  return 0;
}
```

树与图的深度优先遍历

acwing846 树的重心

给定一颗树，树中包含 n 个结点（编号 $1 \sim n$）和 $n-1$ 条无向边。

请你找到树的重心，并输出将重心删除后，剩余各个连通块中点数的最大值。

重心定义：重心是指树中的一个结点，如果将这个点删除后，剩余各个连通块中点数的最大值最小，那么这个节点被称为树的重心。

输入格式

第一行包含整数 n，表示树的结点数。

接下来 $n-1$ 行，每行包含两个整数 a 和 b，表示点 a 和点 b 之间存在一条边。

输出格式

输出一个整数 m，表示将重心删除后，剩余各个连通块中点数的最大值。

数据范围

$1 \leq n \leq 10^5$

cpp

```cpp
#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;

const int N = 100010, M = N * 2;

int n;

int ans = N;

int h[N], e[M], ne[M],  idx;
//idx表示第n条边
//h[N] : 表示 第 i 个节点的 第一条边的 idx
//ne[M] : 表示 与 第 idx 条边 的 下一条边 的 idx
//e[M] : 表示 第idx 条边的 终点

bool st[N]; ·//节点是否被访问过，访问过则标记为true

void add(int a,int b)
{
    e[idx] =  b;            //记录终点节点
    ne[idx] = h[a];        //相当于插入链表 ，将此节点后面的值传进来
    h[a] = idx ++;         //h[a]指向当前新增的节点
}
```

```cpp

int dfs(int u) {
    st[u] = true;    // 标记当前节点已访问

    int sum = 1;     // 当前子树大小（包含自己）
    int res = 0;     // 记录删除 u 后，各连通块的最大节点数

    // 遍历 u 的所有邻接节点
    for (int i = h[u]; i != -1; i = ne[i]) {
        int j = e[i];
        if (!st[j]) {
            int s = dfs(j);    // 递归计算以 j 为根的子树大小
            res = max(res, s); // 更新最大子树大小
            sum += s;          // 累加子树大小
        }
    }

    // 计算父节点方向的连通块大小（总节点数 - 当前子树大小）
    res = max(res, n - sum);

    // 更新全局答案（所有节点的 res 的最小值）
    ans = min(ans, res);

    return sum;
}

int main()
{
    memset (h, -1, sizeof h);
    int a,b;
    cin >> n;
    for (int i = 0;i < n ; i ++)
        {
            cin >> a >>b;
            add(a, b), add(b, a);
        }

    dfs(1);
    cout << ans << endl;
    return 0;
}
```

python

```python
n = int(input().strip())
st = [False for _ in range(n+1)]
graph = {}
res = n
def dfs(u):
```

```python
    global res
    st[u] = True
    size = 0
    sumNode = 0
    for v in graph[u]:
        if not st[v]:
            s = dfs(v)
            sumNode = sumNode+ s
            size = max(size, s) #
    size = max(size, n-sumNode-1)
    res = min(res, size)
    return sumNode+1

for _ in range(n-1):
    a, b = map(int, input().split())
    if a not in graph:      #a到b
        graph[a] = [b]
    else:
        graph[a].append(b)
    if b not in graph:       #b到a
        graph[b] = [a]
    else:
        graph[b].append(a)

dfs(3)
print(res)
```

树与图的广度优先遍历

AcWing 847. 图中点的层次

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环。

所有边的长度都是 1，点的编号为 $1 \sim n$。

请你求出 1 号点到 n 号点的最短距离，如果从 1 号点无法走到 n 号点，输出 -1。

输入格式

第一行包含两个整数 n 和 m。

接下来 m 行，每行包含两个整数 a 和 b，表示存在一条从 a 走到 b 的长度为 1 的边。

输出格式

输出一个整数，表示 1 号点到 n 号点的最短距离。

数据范围

$1 \le n, m \le 10^5$

输入样例：

```
4 5
1 2
2 3
3 4
1 3
1 4
```

输出样例：

```
1
```

```cpp
#include <iostream>
#include <queue>
#include <algorithm>
#include <cstring>
using namespace std;
typedef pair<int,int> PII;
const int N = 100010;
int h[N],ne[N],e[N],idx,st[N];
int dist[N];
int n,m;

int add(int a,int b)
{
    e[idx] = b;
    ne[idx] = h[a];
    h[a] = idx++;
}

void bfs()
{
    memset(dist,0x3f3f3f3f,sizeof dist);
    dist[1] = 0;
    st[1] = 1;
    queue<int> q;
```

```
    q.push(1);
    while(!q.empty())
    {
        int t = q.front();
        q.pop();
        for(int i = h[t];i!=-1;i = ne[i])
        {
            int  j = e[i];
            if(!st[j])
            {
                st[j] = 1;
                dist[j] = dist[t]+1;
                q.push(j);
            }
        }
    }
}
int main()
{
    memset(h,-1,sizeof h);
    cin>>n>>m;
    for(int i = 0;i<m;i++)
    {
        int a,b;
        cin>>a>>b;
        add(a,b);
    }
    bfs();
    if(dist[n]==0x3f3f3f3f)
    cout<<-1;
    else
    cout<<dist[n];
    return 0;
}
```

拓扑序列(有向图)

- **核心要求**：如果存在从 *u* 到 *v* 的路径，则 *u* 必须在 *v* 之前。即“事件执行的合理顺序”
- 有环不存在拓扑序列
- 一个有向**无环图**（拓扑图），一定至少有一个**入度为0的点** -->一定存在拓扑序列
- 使用数组模拟队列输出路径

1. 统计所有节点的**入度**（指向该节点的边数）。
2. 将入度为 0 的节点加入队列。
3. 依次取出队列中的节点，并“删除”它（将其邻接节点的入度减 1）。
4. 若邻接节点入度变为 0，则加入队列。
5. 重复直到队列为空。若所有节点被处理，则得到拓扑序列；否则图中存在环。

AcWing 848. 有向图的拓扑序列

给定一个 n 个点 m 条边的有向图，点的编号是 1 到 n，图中可能存在重边和自环。

请输出任意一个该有向图的拓扑序列，如果拓扑序列不存在，则输出 -1。

若一个由图中所有点构成的序列 A 满足：对于图中的每条边 (x, y)，x 在 A 中都出现在 y 之前，则称 A 是该图的一个拓扑序列。

输入格式

第一行包含两个整数 n 和 m。

接下来 m 行，每行包含两个整数 x 和 y，表示存在一条从点 x 到点 y 的有向边 (x, y)。

输出格式

共一行，如果存在拓扑序列，则输出任意一个合法的拓扑序列即可。

否则输出 -1。

数据范围

$1 \le n, m \le 10^5$

输入样例：

```
3 3
1 2
2 3
1 3
```

输出样例：

```
1 2 3
```

```
#include <cstring>
#include <iostream>
```

```
#include <utility>
using namespace std;
typedef long long LL;
const int N = 1e5 +10;
int arr[N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;

int h[N], e[N], ne[N], idx ;
int q[N], d[N];
int hh = 0,tt = -1;

void add(int a, int b)
{
    e[idx] = b;
    ne[idx] = h[a];
    h[a] = idx ++;
}

void topsort()
{
   for(int i = 1; i<=n;i++)
   {
        if(!d[i])
            q[++tt] = i;    //将入度为0的点入队
   }
   while(hh<=tt)
   {
        int p = q[hh++];
        for(int i = h[p];i!=-1; i = ne[i])
        {
            int j = e[i];
            d[j] --;
            if(d[j] == 0)   //入度减为0
            {
                q[++tt] = j;
            }
        }
   }
   return ;
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m;
    memset(h, -1, sizeof h);
    for (int i = 0; i< m; i++)
    {
        int a, b;
        cin >> a >> b;
```

```
        add(a,b);
        d[b]++; //入度++
    }
    topsort();
    if(tt == n-1)   //所有点入度都减到0
        for(int i = 0; i< n;i++)
        {
            cout << q[i] <<" ";
        }
    else
        cout << -1 << endl;

   // cout << hh << tt << endl;
    return 0;
}
```

最短路问题

单源最短路

Dijkstra是用最短的边去更新其他边，而spfa是上次更新的终点去更新其他边，Bellman-Ford则暴力每次更新所有边

所有边权都为正数

算法	时间复杂度	适用场景	核心思想
朴素Dijkstra	$O(n^2)$	稠密图	每次选择当前距离起点最近的未确定节点，贪心扩展。
堆优化Dijkstra	O(mlogn)	稀疏图	用优先队列维护未确定节点的最小距离，避免线性扫描。

存在负权边

算法	时间复杂度	适用场景	核心思想
Bellman-Ford	O(nm)	任意图	对所有边进行松弛操作，重复 n-1 次保证收敛。
SPFA	平均 O(m)，最坏 O(nm)	负权图但无负环（负环权值之和为负数）	队列优化 Bellman-Ford，仅松弛距离被更新的节点。

多源汇点最短路

Floyd算法　　O(n^3)

- 动态规划：逐步允许通过中间节点 k 更新任意两点间的最短距离。
- 不能处理负环

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，边权可能为负数。

再给定 k 个询问，每个询问包含两个整数 x 和 y，表示查询从点 x 到点 y 的最短距离，如果路径不存在，则输出 `impossible`。

数据保证图中不存在负权回路。

输入格式

第一行包含三个整数 n, m, k。

接下来 m 行，每行包含三个整数 x, y, z，表示存在一条从点 x 到点 y 的有向边，边长为 z。

接下来 k 行，每行包含两个整数 x, y，表示询问点 x 到点 y 的最短距离。

输出格式

共 k 行，每行输出一个整数，表示询问的结果，若询问两点间不存在路径，则输出 `impossible`。

数据范围

$1 \le n \le 200$,
$1 \le k \le n^2$
$1 \le m \le 20000$,
图中涉及边长绝对值均不超过 10000。

输入样例：

```
3 3 2
1 2 1
2 3 2
1 3 1
2 1
1 3
```

输出样例：

```
impossible
1
```

Dijkstra

AcWing 849. Dijkstra求最短路 I

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，所有边权均为正值。

请你求出 1 号点到 n 号点的最短距离，如果无法从 1 号点走到 n 号点，则输出 -1。

输入格式

第一行包含整数 n 和 m。

接下来 m 行每行包含三个整数 x, y, z，表示存在一条从点 x 到点 y 的有向边，边长为 z。

输出格式

输出一个整数，表示 1 号点到 n 号点的最短距离。

如果路径不存在，则输出 -1。

数据范围

$1 \le n \le 500$,
$1 \le m \le 10^5$,
图中涉及边长均不超过10000。

输入样例：

```
3 3
1 2 2
2 3 1
1 3 4
```

输出样例：

```
3
```

朴素Dijkstra算法

1. 将除起点外的距离都置为无穷
2. 选取当前点到另一`未访问`且`距离最短`的边
3. 更新所有点到起点的距离
4. 重复2——3，直到到达终点

```cpp
#include <algorithm>
#include <cstring>
#include <iostream>
using namespace std;
const int N = 510;

int g[N][N]; //为稠密阵所以用邻接矩阵存储
int dist[N]; //用于记录每一个点距离第一个点的距离
bool st[N];  //用于记录该点的最短距离是否已经确定

int n, m;

int Dijkstra() {
  memset(dist, 0x3f, sizeof dist);
  dist[1] = 0;
  for (int i = 0; i < n; i++) //对每个点确定一次最短距离
  {
```

```
    int t = -1;
    for (int j = 1; j <= n; j++) {
      if (!st[j] && (t == -1 || dist[t] > dist[j]))     //选择没有访问且距离最短的下一点
        t = j;
    }
    st[t] = true;       //设置为已访问

    for (int j = 1; j <= n; j++) {
      dist[j] = min(dist[j], dist[t] + g[t][j]);   //更新与t有关点的最短距离
    }
  }

  if (dist[n] == 0x3f3f3f3f)
    return -1; //如果第n个点路径为无穷大即不存在最低路径
  return dist[n];
}

int main() {
  cin >> n >> m;
  memset(g, 0x3f, sizeof g);

  while (m--) {
    int a, b, c;
    cin >> a >> b >> c;
    //自环不会影响Dijkstra
    g[a][b] = min(g[a][b], c);  //重边的话选择最短边
  }
  cout << Dijkstra() << endl;
  return 0;
}
```

AcWing 850. Dijkstra求最短路 II

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，所有边权均为非负值。

请你求出 1 号点到 n 号点的最短距离，如果无法从 1 号点走到 n 号点，则输出 -1。

输入格式

第一行包含整数 n 和 m。

接下来 m 行每行包含三个整数 x, y, z，表示存在一条从点 x 到点 y 的有向边，边长为 z。

输出格式

输出一个整数，表示 1 号点到 n 号点的最短距离。

如果路径不存在，则输出 -1。

数据范围

$1 \le n, m \le 1.5 \times 10^5$，
图中涉及边长均不小于 0，且不超过 10000。
数据保证：如果最短路存在，则最短路的长度不超过 10^9。

输入样例：

```
3 3
1 2 2
2 3 1
1 3 4
```

输出样例：

```
3
```

使用优先队列模拟堆来实现距离的排序，同时再出队时遍历从该点出发的边实现更新最短距离值。

cpp

- 使用小根堆（`priority_queue`）快速获取当前未处理节点中距离最小的节点。
- 相较于朴素版dijkstra算法，堆优化版通过邻接表遍历自动覆盖来处理重边，自环若权值为正则不影响结果。

```cpp
#include <cstring>
#include <iostream>
#include <queue>

using namespace std;

typedef pair<int, int> PII;

const int N = 150010;

// 稀疏图用邻接表来存
int h[N], e[N], ne[N], idx;
int w[N]; // 用来存权重
int dist[N];
bool st[N];
```

```

int n, m;

void add(int a, int b, int c) {p
  e[idx] = b;      //存终点
  ne[idx] = h[a]; //存上一条边索引
  w[idx] = c;      //存权重
  h[a] = idx++;    //存下标
}

int dijkstra() {
  memset(dist, 0x3f, sizeof dist);
  dist[1] = 0;
  priority_queue<PII, vector<PII>, greater<PII>> heap;  //小根堆
  heap.push({0, 1}); // 存入{距离, 节点}, 按距离排序
  while (heap.size()) {
    auto t = heap.top();
    heap.pop();
    int distance = t.first, node = t.second;

    if (st[node])
      continue; // 如果节点被访问过, 则跳过
    st[node] = true;

    for (int i = h[node]; i != -1; i = ne[i]) { //遍历从该节点出发, 所有边
      int j = e[i];
      if (dist[j] > dist[node] + w[i]) {    // 松弛操作
        dist[j] = dist[node] + w[i];
        heap.push({dist[j], j});
      }
    }
  }
  if (dist[n] == 0x3f3f3f3f)
    return -1;
  else
    return dist[n];
}

int main() {
  memset(h, -1, sizeof(h));
  scanf("%d%d", &n, &m);

  while (m--) {
    int x, y, c;
    scanf("%d%d%d", &x, &y, &c);
    add(x, y, c);
  }

  cout << dijkstra() << endl;

  return 0;
}
```

python

```python
import os
import heapq
import sys
import math
sys.setrecursionlimit(10000)

input = lambda: sys.stdin.readline().strip()
n,m = map(int,input().split())

# 防止该点不存在导致程序crash
graph = {i: [] for i in range(1,n+10)}

inf = float(math.inf)

dist = [inf] * (n+10)
st = [False] * (n+10)

def dij():
    hp = []
    heapq.heappush(hp,[0,1])
    while hp:
        dis, point = heapq.heappop(hp)
        if st[point]:
            continue
        st[point] = True

        for v,w in graph[point]:
            if dist[v] > dis + w:
                dist[v] = dis + w
                heapq.heappush(hp,(dist[v],v))


for i in range(m):
    x,y,z = map(int,input().split())
    graph[x].append((y,z))

dij()
if dist[n] == inf:
    print(-1)
else:
    print(dist[n])

```

bellman-ford （有边数限制）

AcWing 853. 有边数限制的最短路

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环， **边权可能为负数。**

请你求出从 1 号点到 n 号点的最多经过 k 条边的最短距离，如果无法从 1 号点走到 n 号点，输出 `impossible` 。

注意：图中可能 **存在负权回路** 。

输入格式

第一行包含三个整数 n, m, k。

接下来 m 行，每行包含三个整数 x, y, z，表示存在一条从点 x 到点 y 的有向边，边长为 z。

点的编号为 $1 \sim n$。

输出格式

输出一个整数，表示从 1 号点到 n 号点的最多经过 k 条边的最短距离。

如果不存在满足条件的路径，则输出 `impossible` 。

数据范围

$1 \le n, k \le 500$,
$1 \le m \le 10000$,
$1 \le x, y \le n$,
任意边长的绝对值不超过 10000。

输入样例：

```
3 3 1
1 2 1
2 3 1
1 3 3
```

输出样例：

```
3
```

连续进行松弛，在每次松弛时把每条边都更新一下。

若在 n-1 次松弛后还能更新，则说明图中有负环，因此无法得出结果。

```
#include<iostream>
#include<cstring>

using namespace std;

const int N = 510, M = 10010;

struct Edge {
    int a;
```

```c
    int b;
    int w;
} e[M];//把每个边保存下来即可
int dist[N];
int back[N];//备份数组防止串联
int n, m, k;//k代表最短路径最多包含k条边


void bellman_ford() {
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    for(int i = 0; i < k; i++)
    {
        memcpy(back, dist, sizeof dist);
        //对于每个点，都需要对每条边进行松弛操作
        //我们需要把i-1次的迭代结果保存下来，保每次迭代只基于上一轮的结果（）
        //A→B→C，若直接更新 dist，可能在同一轮中同时更新 B 和 C，违反“最多经过 k 条边”的限制。
        for (int i = 0; i < m; i++) {
            int a = e[i].a, b = e[i].b, w = e[i].w;
            dist[b] = min(dist[b], back[a] + w);        //使用备份更新最短距离
        }
    }
}


int main() {
    scanf("%d%d%d", &n, &m, &k);
    for (int i = 0; i < m; i++) {
        int a, b, w;
        scanf("%d%d%d", &a, &b, &w);
        e[i] = {a, b, w};
    }
    bellman_ford();

    if(dist[n]>0x3f3f3f3f/2) //若无法到达n号点，但x,n存在且为负值，则在每次枚举点时都会更新n的距离，使得距离小于0x3f3f3f3f
        puts("impossible");
    else printf("%d",dist[n]);

    return 0;
}
```

spfa

AcWing 851. spfa求最短路

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环， **边权可能为负数。**

请你求出 1 号点到 n 号点的最短距离，如果无法从 1 号点走到 n 号点，则输出 `impossible`。

数据保证不存在负权回路。

输入格式

第一行包含整数 n 和 m。

接下来 m 行每行包含三个整数 x, y, z，表示存在一条从点 x 到点 y 的有向边，边长为 z。

输出格式

输出一个整数，表示 1 号点到 n 号点的最短距离。

如果路径不存在，则输出 `impossible`。

数据范围

$1 \le n, m \le 10^5$,
图中涉及边长绝对值均不超过 10000。

输入样例：

```
3 3
1 2 5
2 3 -3
1 3 4
```

输出样例：

```
2
```

- 只有当一个点的前驱结点更新了，该节点才会得到更新 -->创建一个队列每一次加入距离被更新的结点。
- st数组的作用：判断当前的点是否已经加入到队列中，减少不必要的遍历
- 有负权回路SPFA会死循环

```

#include <cstring>
#include <iostream>
#include <algorithm>
#include <queue>

using namespace std;

const int N = 100010;

int n, m;
int h[N], w[N], e[N], ne[N], idx;
int dist[N];
bool st[N];

```

```
void add(int a, int b, int c){
    e[idx] = b;
    ne[idx] = h[a];
    w[idx] = c;
    h[a] = idx++;
}


int spfa(){
    memset(dist, 0x3f, sizeof dist);
    queue<int> q;
    q.push(1);
    dist[1] = 0;
    while(q.size())
    {
        int t = q.front();
        q.pop();
        st[t] = false;
        for(int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if(dist[j] > dist[t] + w[i])
            {
                dist[j] = dist[t] + w[i];
                if(!st[j])  ///当前已经加入队列的结点，无需再次加入队列
                    {
                        st[j] = true;
                        q.push(j);
                    }
            }
        }
    }
    return dist[n];
}



int main(){
    memset(h, -1, sizeof h);
    memset(dist, 0x3f, sizeof dist);
    int n, m;
    cin >> n >> m;
    for(int i = 0; i < m; i++){
        int a, b, w;
        cin >> a >> b >> w;
        add(a, b, w);
    }
    spfa();
    if(dist[n] == 0x3f3f3f3f )//如果到n点的距离是无穷，则不能到达,因为spfa只遍历相连的边，而Bellman_ford遍历所有边
        cout << "impossible";
    else cout << dist[n];
    return 0;
```

```
}

```

AcWing 852. spfa判断负环

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环， **边权可能为负数。**

请你判断图中是否存在负权回路。

输入格式

第一行包含整数 n 和 m。

接下来 m 行每行包含三个整数 x, y, z，表示存在一条从点 x 到点 y 的有向边，边长为 z。

输出格式

如果图中**存在**负权回路，则输出 `Yes`，否则输出 `No`。

数据范围

$1 \le n \le 2000$,
$1 \le m \le 10000$,
图中涉及边长绝对值均不超过 10000。

输入样例：

```
3 3
1 2 -1
2 3 4
3 1 -4
```

输出样例：

```
Yes
```

统计当前每个点的最短路中所包含的边数，如果某点的最短路所包含的边数大于等于n，则也说明存在环

spfa

```
#include <algorithm>
#include <cstring>
#include <iostream>
#include <queue>

using namespace std;

const int N = 2e3 + 10, M = 1e4 + 10;

int n, m;
```

```cpp
int h[N], e[M], ne[M], w[M], idx;
bool st[N];      // 是否在队列中
int dist[N];     // 到起点的最短距离
int cnt[N];      // 最短路径的边数

void add(int a, int b, int c) {
    e[idx] = b;
    w[idx] = c;
    ne[idx] = h[a];
    h[a] = idx++;
}

bool spfa() {
    queue<int> q;
    // 初始化：所有节点入队，防止非连通图漏判
    for (int i = 1; i <= n; i++) {
        q.push(i);
        st[i] = true;
    }

    while (!q.empty()) {
        int t = q.front();
        q.pop();
        st[t] = false;

        for (int i = h[t]; i != -1; i = ne[i]) {
            int j = e[i];
            if (dist[j] > dist[t] + w[i]) {
                dist[j] = dist[t] + w[i];
                cnt[j] = cnt[t] + 1;

                // 根据 Bellman-Ford，最短路径最多 n-1 条边
                // 如果 cnt[j] >= n，说明存在负环
                if (cnt[j] >= n)
                    return true;

                // 如果 j 不在队列中，则入队
                if (!st[j]) {
                    q.push(j);
                    st[j] = true;
                }
            }
        }
    }

    return false;
}

int main() {
    memset(h, -1, sizeof h);
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b, c;
```

```
        cin >> a >> b >> c;
        add(a, b, c);
    }

    if (spfa())
        cout << "Yes";
    else
        cout << "No";

    return 0;
}
```

bellman-ford

```
#include <bits/stdc++.h>
using namespace std;

const int N = 2005,M = 1e4 + 5,INF = 0x3f3f3f3f;
struct edge
{
    int a,b,c;
}e[M];
int n,m,dis[N];

string bellman()
{
    memset(dis,0x3f,sizeof dis);
    dis[1] = 0;

    bool flag = true;
    int cnt = 0;
    while(flag)
    {
        flag = false;
        cnt++;
        if(cnt > n) return "Yes";  //迭代超过n次根据抽屉原理出现了n+1个点既出现了负环
        for(int i=0;i<m;i++)
        {
            int a = e[i].a,b = e[i].b,c = e[i].c;
            if(dis[b] > dis[a] + c)
            {
                flag = true;
                dis[b] = dis[a] + c;
            }
        }
    }
    return "No";
}
int main()
```

```
{
    cin>>n>>m;
    for(int i=0;i<m;i++)
    {
        int a,b,c;
        cin>>a>>b>>c;
        e[i] = {a,b,c};
    }
    cout<<bellman()<<endl;
    return 0;
}

```

floyd

Floyd-Warshall 算法的动态规划状态转移方程可以表示为：

- $d^{(k)}[i][j] = \min\left(d^{(k-1)}[i][j],\ d^{(k-1)}[i][k] + d^{(k-1)}[k][j]\right)$
- $d^{(k)}[i][j]$：允许使用前 k 个节点作为中转时，节点 i 到 j 的最短距离。

状态转移：

- **不经过 k**：保持 $d^{(k-1)}[i][j]$
- **经过 k**：路径分解为 $i \to k$ 和 $k \to j$ 的最短距离之和 $d^{(k-1)}[i][k] + d^{(k-1)}[k][j]$

滚动数组优化：

`d[i][j] ← min(d[i][j], d[i][k] + d[k][j])`

- 通过就地更新，将三维 状态压缩为二维。

AcWing 854. Floyd求最短路

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，边权可能为负数。

再给定 k 个询问，每个询问包含两个整数 x 和 y，表示查询从点 x 到点 y 的最短距离，如果路径不存在，则输出 `impossible`。

数据保证图中不存在负权回路。

输入格式

第一行包含三个整数 n, m, k。

接下来 m 行，每行包含三个整数 x, y, z，表示存在一条从点 x 到点 y 的有向边，边长为 z。

接下来 k 行，每行包含两个整数 x, y，表示询问点 x 到点 y 的最短距离。

输出格式

共 k 行，每行输出一个整数，表示询问的结果，若询问两点间不存在路径，则输出 `impossible`。

数据范围

$1 \le n \le 200$,
$1 \le k \le n^2$
$1 \le m \le 20000$,
图中涉及边长绝对值均不超过 10000。

输入样例：

```
3 3 2
1 2 1
2 3 2
1 3 1
2 1
1 3
```

输出样例：

```
impossible
1
```

```cpp
#include <iostream>
#include <utility>
using namespace std;
typedef long long LL;
const int N = 2e4 +10;
int d[N][N];
int INF = 1e9;

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m,Q;

void floyd()
{
    for(int k = 1; k <= n; k++)
```

```
        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m >> Q;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
            if(i == j) d[i][j] = 0;
            else d[i][j] = INF;
    while(m--)
    {
        int a,b,c;
        cin >> a >> b >> c;
        d[a][b] = min(d[a][b], c);   // 处理重边
    }
    floyd();
    while(Q--)
    {
        int a,b;
        cin >> a >> b;
        if(d[a][b] > INF/2) cout << "impossible" << endl;
        else cout << d[a][b] << endl;
    }
}

```

最小生成树

- 普利姆算法（Prim）
    - 朴素版Prim　O(n^2)　　稠密图
    - 堆优化版Prim　O(mlogn)
- 克鲁斯卡尔算法（Kruskal）　O(mlogm)　稀疏图

Prim

AcWing858. Prim算法求最小生成树

给定一个 n 个点 m 条边的无向图，图中可能存在重边和自环，边权可能为负数。

求最小生成树的树边权重之和，如果最小生成树不存在则输出 `impossible`。

给定一张边带权的无向图 $G=(V,E)$，其中 V 表示图中点的集合，E 表示图中边的集合，$n=|V|$，$m=|E|$。

由 V 中的全部 n 个顶点和 E 中 $n-1$ 条边构成的无向连通子图被称为 G 的一棵生成树，其中边的权值之和最小的生成树被称为无向图 G 的最小生成树。

输入格式

第一行包含两个整数 n 和 m。

接下来 m 行，每行包含三个整数 u,v,w，表示点 u 和点 v 之间存在一条权值为 w 的边。

输出格式

共一行，若存在最小生成树，则输出一个整数，表示最小生成树的树边权重之和，如果最小生成树不存在则输出 `impossible`。

数据范围

$1 \le n \le 500$,
$1 \le m \le 10^5$,
图中涉及边的边权的绝对值均不超过 10000。

输入样例：

```
4 5
1 2 1
1 3 2
1 4 3
2 3 2
3 4 4
```

输出样例：

```
6
```

Prim算法：我们从其中一个点出发，遍历该点所有边，找到最小权值的边，然后加入集合，更新该边另一顶点引出的边（使从该边引出的节点可达）。

```cpp
#include <cstring>
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 510, INF = 0x3f3f3f3f;

int n, m;
int g[N][N];
int dist[N];
bool st[N]; // 记录每个节点是否已经被包含在最小生成树中


```

```
int prim()
{
    memset(dist, 0x3f, sizeof dist);
    int res = 0;
    for(int i = 0; i < n; i ++ )
    {
        int t = -1;
        for(int j = 1; j <= n; j ++ )
            if(!st[j] && (t == -1 || dist[t] > dist[j]))    //贪心，找到距离当前集合的最小边
                t = j;
        if(i && dist[t] == INF) return INF;  //点不可达，不能生成最小树

        // 更新必须放在for循环前,避免自环，且当t放入集合后，其dist[t]已经没有意义
        if(i)   res += dist[t]; // 如果不是第一个点，将其权值加入结果中
        st[t] = true;

        for(int j = 1; j <= n; j ++ )
            dist[j] = min(dist[j], g[t][j]);        // 更新 dist 数组，记录每个节点到当前生成树的最小距离
        //即将一些点变为可达的点
    }
    return res;
}


int main()
{
    scanf("%d%d", &n, &m);

    memset(g, 0x3f, sizeof g);

    while (m -- )
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        g[a][b] = g[b][a] = min(g[a][b], c);    //取最短边，Prim算法针对无向图，因此赋值两次
    }

    int t = prim();

    if (t == INF) puts("impossible");
    else printf("%d\n", t);

    return 0;
}

```

[AcWing 858. Prim算法求最小生成树：图解+详细代码注释（带上了保存路径） - AcWing]

```
#include <iostream>
#include <cstring>
#include <algorithm>
using namespace std;
```

```cpp

const int N = 510;
int g[N][N];//存储图
int dt[N];//存储各个节点到生成树的距离
int st[N];//节点是否被加入到生成树中
int pre[N];//节点的前去节点
int n, m;//n 个节点，m 条边

void prim()
{
    memset(dt,0x3f, sizeof(dt));//初始化距离数组为一个很大的数（10亿左右）
    int res= 0;
    dt[1] = 0;//从 1 号节点开始生成
    for(int i = 0; i < n; i++)//每次循环选出一个点加入到生成树
    {
        int t = -1;
        for(int j = 1; j <= n; j++)//每个节点一次判断
        {
            if(!st[j] && (t == -1 || dt[j] < dt[t]))//如果没有在树中，且到树的距离最短，则选择该点
                t = j;
        }

        //2022.6.1 发现测试用例加强后，需要判断孤立点了
        //如果孤立点，直返输出不能，然后退出
        if(dt[t] == 0x3f3f3f3f) {
            cout << "impossible";
            return;
        }


        st[t] = 1;// 选择该点
        res += dt[t];
        for(int i = 1; i <= n; i++)//更新生成树外的点到生成树的距离
        {
            if(dt[i] > g[t][i] && !st[i])//从 t 到节点 i 的距离小于原来距离，则更新。
            {
                dt[i] = g[t][i];//更新距离
                pre[i] = t;//从 t 到 i 的距离更短，i 的前驱变为 t.
            }
        }
    }

    cout << res;

}

void getPath()//输出各个边
{
    for(int i = n; i > 1; i--)//n 个节点，所以有 n-1 条边。

    {
```

```
        cout << i <<" " << pre[i] << " "<< endl;// i 是节点编号，pre[i] 是 i 节点的前驱节点。他们构成一条边。
    }
}

int main()
{
    memset(g, 0x3f, sizeof(g));//各个点之间的距离初始化成很大的数
    cin >> n >> m;//输入节点数和边数
    while(m --)
    {
        int a, b, w;
        cin >> a >> b >> w;//输出边的两个顶点和权重
        g[a][b] = g[b][a] = min(g[a][b],w);//存储权重
    }

    prim();//求最下生成树
    //getPath();//输出路径
    return 0;
}


作者：Hasity
链接：https://www.acwing.com/solution/content/38312/
来源：AcWing
著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。
```

AcWing 859. Kruskal算法求最小生成树

859. Kruskal算法求最小生成树

题目 提交记录 讨论 题解 视频讲解

给定一个 n 个点 m 条边的无向图，图中可能存在重边和自环，边权可能为负数。

求最小生成树的树边权重之和，如果最小生成树不存在则输出 `impossible`。

给定一张边带权的无向图 $G=(V,E)$，其中 V 表示图中点的集合，E 表示图中边的集合，$n=|V|$，$m=|E|$。

由 V 中的全部 n 个顶点和 E 中 $n-1$ 条边构成的无向连通子图被称为 G 的一棵生成树，其中边的权值之和最小的生成树被称为无向图 G 的最小生成树。

输入格式

第一行包含两个整数 n 和 m。

接下来 m 行，每行包含三个整数 u,v,w，表示点 u 和点 v 之间存在一条权值为 w 的边。

输出格式

共一行，若存在最小生成树，则输出一个整数，表示最小生成树的树边权重之和，如果最小生成树不存在则输出 `impossible`。

数据范围

$1\le n\le 10^5$,
$1\le m\le 2*10^5$,
图中涉及边的边权的绝对值均不超过 1000。

输入样例：

```
4 5
1 2 1
1 3 2
1 4 3
2 3 2
3 4 4
```

输出样例：

```
6
```

kruskal算法：

通过并查集对边进行合并。先按照从小到大的权重进行排序，然后对每两个未连接的节点进行连边。我们知道，对于一个无向图，每两点有且仅有一条边相连，若其边数小于n-1，则其不联通，不能构成最小生成树。

```
#include <iostream>
#include <cstring>
#include <algorithm>

```

```
using  namespace std;

const int N = 100010, M = 200010, INF = 0x3f3f3f3f;

int n, m;
int p[N];

struct Edge {
    int a, b, w;
    bool operator<(const Edge &E) const{
        return w < E.w;
    }
} edges[M];



int find(int x){        //并查集
    if(p[x] != x) p[x] = find(p[x]);
    return p[x];
}

int kruskal(){
    sort(edges, edges + m);
    for (int i = 1; i <= n; i ++ ) p[i] = i;    //初始化并查集
    int res = 0, cnt = 0;
    for (int i = 0; i < m; i ++ )
    {
        int a = edges[i].a, b = edges[i].b, w = edges[i].w;
        a = find(a), b = find(b);
        if (a != b)
        {
            p[a] = b;
            res += w;
            cnt ++ ;
        }
    }
    if (cnt < n - 1) return INF;        // 如果边的数量不足 n-1，说明图不连通
    else return res;
}


int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i ++ )
    {
        int a, b, w;
        scanf("%d%d%d", &a, &b, &w);
        edges[i] = {a, b, w};
    }

    int t = kruskal();
```

```

    if (t == INF) puts("impossible");
    else printf("%d\n", t);

    return 0;
}
```

二分图

- 染色法　O(n+m)
- 匈牙利算法　　O(mn)，实际运行时间很小

- **二分图**：二分图是一种特殊的图，其顶点集可以分成两个互不相交的子集，使得每一条边的两个端点分别属于不同的子集。（集合内部没有边）
- **二部**：如果一个图是二分图，那么它没有奇数长度的环。
- **着色**：一个图是二分图当且仅当它可以被 2着色，即可以使用两种颜色对图的顶点进行着色，使得每条边的两个端点的颜色不同。

染色法

AcWing 860. 染色法判定二分图

给定一个 n 个点 m 条边的无向图，图中可能存在重边和自环。

请你判断这个图是否是二分图。

输入格式

第一行包含两个整数 n 和 m。

接下来 m 行，每行包含两个整数 u 和 v，表示点 u 和点 v 之间存在一条边。

输出格式

如果给定图是二分图，则输出 `Yes`，否则输出 `No`。

数据范围

$1 \le n, m \le 10^5$

输入样例：

```
4 4
1 3
1 4
2 3
2 4
```

输出样例：

```
Yes
```

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100010 * 2;
int e[N], ne[N], idx;//邻接表存储图
int h[N];
int color[N];//保存各个点的颜色，0 未染色，1 是红色，2 是黑色
int n, m;//点和边

void add(int a,int b){
    e[idx] = b;
    ne[idx] = h[a];
    h[a] = idx++;
}

bool dfs(int u, int c)
{
    color[u] = c;//u的点成 c 染色

    for(int i = h[u]; i!= -1; i = ne[i])
```

```
    {
        int b = e[i];
        if(!color[b])
        {
            if(!dfs(b, 3 - c)) return false;//（3 - 1 = 2，  如果 u 的颜色是2，则和 u 相邻的染成 1）
                                                //（3 - 2 = 1，  如果 u 的颜色是1，则和 u 相邻的染成 2）
        }
        else if(color[b] == c)//如果已经染色，判断颜色是否为  c
        {
            return false;
        }
    }
    return true;
}



int main(){
    memset(h, -1, sizeof h);
    scanf("%d%d", &n, &m);
    for(int i = 0;i < m; i ++)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        add(a, b);
        add(b, a);
    }
     for(int i = 1; i <= n; i++)//遍历点
    {
        if(!color[i])//如果没染色
        {
            if(!dfs(i, 1))//染色该点，并递归处理和它相邻的点
            {
                cout << "No" << endl;
                return 0;
            }

        }
    }
    cout << "Yes" << endl;
    return 0;

}
```

匈牙利算法：

匈牙利算法是一种用于解决指派问题（Assignment Problem）的数学算法。指派问题是指在有限的候选者中选择一组匹配，使得总的成本最小（或收益最大）。

AcWing 861. 二分图的最大匹配

给定一个二分图，其中左半部包含 n_1 个点（编号 $1 \sim n_1$），右半部包含 n_2 个点（编号 $1 \sim n_2$），二分图共包含 m 条边。

数据保证任意一条边的两个端点都不可能在同一部分中。

请你求出二分图的最大匹配数。

> 二分图的匹配：给定一个二分图 G，在 G 的一个子图 M 中，M 的边集 $\{E\}$ 中的任意两条边都不依附于同一个顶点，则称 M 是一个匹配。
>
> 二分图的最大匹配：所有匹配中包含边数最多的一组匹配被称为二分图的最大匹配，其边数即为最大匹配数。

输入格式

第一行包含三个整数 n_1、 n_2 和 m。

接下来 m 行，每行包含两个整数 u 和 v，表示左半部点集中的点 u 和右半部点集中的点 v 之间存在一条边。

输出格式

输出一个整数，表示二分图的最大匹配数。

数据范围

$1 \le n_1, n_2 \le 500$,
$1 \le u \le n_1$,
$1 \le v \le n_2$,
$1 \le m \le 10^5$

输入样例：

```
2 2 4
1 1
1 2
2 1
2 2
```

输出样例：

```
2
```

`find(int u)`：尝试为左边节点 `u` 找到一个匹配，使用递归方式实现深度优先搜索（DFS）。

如果右边节点 `j` 未被访问过，则标记为已访问。

如果右边节点 `j` 未匹配或者其匹配的左边节点可以重新匹配，则更新匹配关系并返回 `true`。

```cpp
#include <iostream>
#include <cstring>
using namespace std;

const int N = 505, M = 1e5 + 10;

int n1,n2, m;
int h[N], e[M], ne[M], idx;

int match[N], st[N];    //st标记右半部分


void add(int a, int b){
    e[idx] = b;
    ne[idx] = h[a];
    h[a] = idx++;
}

int find(int u) // 匈牙利算法
{
    for(int i = h[u]; i != -1; i = ne[i]){
        int j = e[i];
        if(!st[j]){
            st[j] = true;      //标记该节点被占用，后续调用不能使用该节点
            if(match[j] == 0 || find(match[j])){    //如果该节点没有被匹配，则匹配
                                                    //若该节点已匹配，则递归调用重新匹配节点
                match[j] = u;
                return true;
            }
        }
    }
    return false;
}

int main(){
    memset(h, -1, sizeof h);
    scanf("%d%d%d", &n1, &n2, &m);
    for(int i = 0; i < m; i++){
        int a, b;
        scanf("%d%d", &a, &b);
        add(a, b);  // 保存图，因为只从一遍找另一边，所以该无向图只需要存储一个方向
    }
     int res = 0;
    //为各个点找匹配
    for(int i = 1; i <= n1; i++){
        memset(st, 0, sizeof st);   //将st数组置0，使得所有节点都没有被标记
        if(find(i)) res++;
    }
    cout << res;
    return 0;
}

```

提高课

- **普通 BFS**：从单个起点出发，逐层扩展。
- **多源 BFS**：从多个起点同时出发，逐层扩展。
    - 取一虚拟原点，将所有起点以边权为0连接到虚拟原点，则转化为单源BFS

多源BFS

AcWing 173. 矩阵距离

给定一个 N 行 M 列的 01 矩阵 A，$A[i][j]$ 与 $A[k][l]$ 之间的曼哈顿距离定义为：

$$dist(i,j,k,l)=|i-k|+|j-l|$$

输出一个 N 行 M 列的整数矩阵 B，其中：

$$B[i][j]=min_{1\le x\le N,1\le y\le M,A[x][y]=1}dist(i,j,x,y)$$

输入格式

第一行两个整数 N,M。

接下来一个 N 行 M 列的 01 矩阵，数字之间没有空格。

输出格式

一个 N 行 M 列的矩阵 B，相邻两个整数之间用一个空格隔开。

数据范围

$1\le N,M\le 1000$

输入样例：

```
3 4
0001
0011
0110
```

输出样例：

```
3 2 1 0
2 1 0 0
1 0 0 1
```

```
#include <cstring>
```

```cpp
#include <iostream>
#include <queue>
#include <utility>
using namespace std;
typedef long long LL;
const int N = 1e3 +10;
int arr[N];
char g[N][N];
int d[N][N];


typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;
int xway[4] = {-1, 1, 0, 0};
int yway[4] = {0, 0, -1, 1};
void bfs()
{
    queue<pair<int, int>> qu;
    memset(d, -1, sizeof d);
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
    {
        if(g[i][j] == '1')
        {
            qu.push({i, j});
            d[i][j] = 0;

       }
    }
    while(!qu.empty())
    {
        auto t = qu.front();
        qu.pop();
        for(int i = 0; i < 4; i++)
        {
            int x = t.first + xway[i];
            int y = t.second + yway[i];
            if(x >= 0 && x < n && y >= 0 && y < m && d[x][y] == -1)
            {
                d[x][y] = d[t.first][t.second] + 1;
                qu.push({x, y});
            }
        }
    }
}
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m;
    for(int i = 0 ; i < n ; i ++)
        cin>>g[i];
    bfs();
```

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cout << d[i][j] << ' ';
        }
        cout << "\n";
    }
}

```

DFS之连通性模型

AcWing 1112. 迷宫

一天Extense在森林里探险的时候不小心走入了一个迷宫，迷宫可以看成是由 $n*n$ 的格点组成，每个格点只有2种状态，`.` 和 `#`，前者表示可以通行后者表示不能通行。

同时当Extense处在某个格点时，他只能移动到东南西北(或者说上下左右)四个方向之一的相邻格点上，Extense想要从点A走到点B，问在不走出迷宫的情况下能不能办到。

如果起点或者终点有一个不能通行(为#)，则看成无法办到。

注意：A、B不一定是两个不同的点。

输入格式

第1行是测试数据的组数 k，后面跟着 k 组输入。

每组测试数据的第1行是一个正整数 n，表示迷宫的规模是 $n*n$ 的。

接下来是一个 $n*n$ 的矩阵，矩阵中的元素为 `.` 或者 `#`。

再接下来一行是 4 个整数 h_a,l_a,h_b,l_b，描述 A 处在第 h_a 行, 第 l_a 列，B 处在第 h_b 行, 第 l_b 列。

注意到 h_a,l_a,h_b,l_b 全部是从 0 开始计数的。

输出格式

k行，每行输出对应一个输入。

能办到则输出“YES”，否则输出“NO”。

数据范围

$1 \le n \le 100$

输入样例:

```
2
3
.##
..#
#..
0 0 2 2
5
.....
###.#
..#..
###..
...#.
0 0 4 0
```

输出样例:

```
YES
NO
```

```
#include <iostream>
#include <utility>
#include <cstring>
using namespace std;
typedef long long LL;
```

```
const int N = 1e3 +10;
bool d[N][N];
bool st[N][N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;

int xway[4] = {-1, 1, 0, 0};
int yway[4] = {0, 0, -1, 1};
int sx,sy,ex,ey;



bool dfs(int x, int y) {
    if(x==ex && y==ey)          //判断终点
        return true;
    st[x][y] = true;
    for(int i = 0; i < 4; i++)
    {
        int nx = x + xway[i];
        int ny = y + yway[i];
        if(nx>=1 && ny>=1 && nx<=m && ny<=m && !st[nx][ny] && d[nx][ny]!=1)
        {
            if(dfs(nx,ny)) return true;     //直接返回到solve
        }
    }
    return false;
}

void solve() {
    cin >> m;
    for(int i = 1; i <= m; i++)
     for(int j = 1; j <= m; j++)
    {
        char ch;
        cin >> ch;
        if(ch=='#') d[i][j] = true;
        else d[i][j] = false;
    }
    memset(st,false,sizeof st);
    cin >> sx >> sy >> ex >> ey;
    sx++;
    sy++;
    ey++;
    ex++;
    if(d[sx][sy] || d[ex][ey])
    {
        cout << "NO" << endl;    //特判
    }
    else {
        if(dfs(sx,sy)) cout << "YES" << endl;
        else cout << "NO" << endl;
```

```
    }
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n;
    while(n--) {
        solve();
    }
}
```

AcWing 1113. 红与黑

有一间长方形的房子，地上铺了红色、黑色两种颜色的正方形瓷砖。

你站在其中一块黑色的瓷砖上，只能向相邻（上下左右四个方向）的黑色瓷砖移动。

请写一个程序，计算你总共能够到达多少块黑色的瓷砖。

输入格式

输入包括多个数据集合。

每个数据集合的第一行是两个整数 W 和 H，分别表示 x 方向和 y 方向瓷砖的数量。

在接下来的 H 行中，每行包括 W 个字符。每个字符表示一块瓷砖的颜色，规则如下

1）‘.’：黑色的瓷砖；
2）‘#’：红色的瓷砖；
3）‘@’：黑色的瓷砖，并且你站在这块瓷砖上。该字符在每个数据集合中唯一出现一次。

当在一行中读入的是两个零时，表示输入结束。

输出格式

对每个数据集合，分别输出一行，显示你从初始位置出发能到达的瓷砖数(记数时包括初始位置的瓷砖)。

数据范围

$1 \le W, H \le 20$

输入样例：

```
6 9
....#.
.....#
......
......
......
......
......
#@...#
.#..#.
0 0
```

输出样例：

```
45
```

BFS

```
#include <iostream>
#include <utility>
#include <queue>
#include <cstring>
using namespace std;
typedef long long LL;
```

```
const int N = 1e3 +10;
bool arr[N][N];
bool st[N][N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;

int sx,sy;

int dx[] = {-1,0,0,1};
int dy[] = {0,-1,1,0};
int BFS()
{
    int res =1;
    queue<PA> q;
    st[sx][sy] = true;
    q.push({sx,sy});

    while(q.size())
    {
        auto tmp = q.front();
        sx = tmp.first;
        sy = tmp.second;
        q.pop();
        for(int i = 0;i < 4;i++)
        {

            int xx = sx + dx[i];
            int yy = sy + dy[i];
            if(xx < 1 || xx> m || yy < 1 || yy > n)
            {
                //cout << xx << " " << yy << endl;
                //cout << endl;
                continue;
            }

            if(arr[xx][yy] && !st[xx][yy])
            {
                q.push({xx,yy});

                res++;
                //cout << 2323<<endl;
                st[xx][yy] = true;

            }
        }
    }

    return res;
}

int main() {
```

```
    ios::sync_with_stdio(0);
    cin.tie(0);
    while(1)
    {
        memset(st,0,sizeof st);
        cin >> n >> m;
        if(n==0 && m==0)    return 0;
        for(int i = 1;i <= m;i++)
        {
            for(int j = 1;j <= n;j++)
            {
                 char tmp;
                 cin >> tmp;
                 if(tmp == '.')
                    arr[i][j] = true;
                 else if(tmp == '@')
                    {
                        sx = i;
                        sy = j;
                        arr[i][j] = true;
                    }
                 else
                 {
                    arr[i][j] = false;

                 }
          //      cout << arr[i][j] << " ";
            }
       //     cout << endl;
        }

        cout << BFS() << endl;


    }
}

```

DFS

```
#include <iostream>
#include <utility>
#include <cstring>
using namespace std;
typedef long long LL;
const int N = 1e3 +10;
bool arr[N][N];
bool st[N][N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;
int sx,sy;
```

```


int dx[] = {-1,0,0,1};
int dy[] = {0,-1,1,0};

int dfs(int x, int y)
{
    int res = 0;
    st[x][y] = true;
    for(int i = 0; i<4;i++)
    {
        int xx = x + dx[i];
        int yy = y + dy[i];
        if(xx < 1 || xx > m || yy < 1 || yy > n)
            continue;
        if(arr[xx][yy] && !st[xx][yy])
        {
            res+=  dfs(xx,yy);
        }
    }
    return res + 1;
}
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    while(1)
    {
        memset(st,0,sizeof st);
        cin >> n >> m;
        if(n == 0 && m == 0)
            break;
        for(int i = 1; i<=m;i++)
            for(int j = 1; j<=n;j++)
        {
            char tmp;
            cin >> tmp;
            if(tmp == '.')
               arr[i][j] = true;
            else if(tmp == '@')
               {
                   sx = i;
                   sy = j;
                   arr[i][j] = true;
               }
            else
            {
               arr[i][j] = false;
            }

        }

        cout << dfs(sx,sy) << endl;
    }
```

```

}

```

DFS之搜索顺序

AcWing 1116. 马走日

马在中国象棋以日字形规则移动。

请编写一段程序，给定 $n*m$ 大小的棋盘，以及马的初始位置 $(x,\ y)$，要求不能重复经过棋盘上的同一个点，计算马可以有多少途径遍历棋盘上的所有点。

输入格式

第一行为整数 T，表示测试数据组数。

每一组测试数据包含一行，为四个整数，分别为棋盘的大小以及初始位置坐标 n,m,x,y。

输出格式

每组测试数据包含一行，为一个整数，表示马能遍历棋盘的途径总数，若无法遍历棋盘上的所有点则输出 0。

数据范围

$1 \le T \le 9$,
$1 \le m,n \le 9$,
$1 \le n \times m \le 28$,
$0 \le x \le n-1$,
$0 \le y \le m-1$

输入样例：

```
1
5 4 0 0
```

输出样例：

```
32
```

```
#include <iostream>
#include <cstring>
using namespace std;
const int N = 10;
int t,n,m,sx,sy,ans = 0;
int dx[] = {1,-1,2,-2,1,-1,2,-2};
int dy[] = {2,2,1,1,-2,-2,-1,-1};
bool vis[N][N];
void dfs (int x,int y,int t) {
    if (t == n * m) {
        ans++;
        return ;
    }
    for (int i = 0;i < 8;i++) {
```

```
        int a = x + dx[i],b = y + dy[i];
        if (a < 0 || a >= n || b < 0 || b >= m || vis[a][b]) continue;
        vis[a][b] = true;
        dfs (a,b,t + 1);
        vis[a][b] = false;
    }
}
int main () {
    cin >> t;
    while (t--) {
        ans = 0;
        memset (vis,false,sizeof (vis));
        cin >> n >> m >> sx >> sy;

        vis[sx][sy] = true;
        dfs (sx,sy,1);
        cout << ans << endl;
    }
    return 0;
}

```

AcWing 1117. 单词接龙

单词接龙是一个与我们经常玩的成语接龙相类似的游戏。

现在我们已知一组单词，且给定一个开头的字母，要求出以这个字母开头的最长的“龙”，每个单词最多被使用两次。

在两个单词相连时，其重合部分合为一部分，例如 beast 和 astonish ，如果接成一条龙则变为 beastonish。

我们可以任意选择重合部分的长度，但其长度必须大于等于1，且严格小于两个串的长度，例如 at 和 atide 间不能相连。

输入格式

输入的第一行为一个单独的整数 n 表示单词数，以下 n 行每行有一个单词（只含有大写或小写字母，长度不超过20），输入的最后一行为一个单个字符，表示“龙”开头的字母。

你可以假定以此字母开头的“龙”一定存在。

输出格式

只需输出以此字母开头的最长的“龙”的长度。

数据范围

$n \leq 20$，
单词随机生成。

输入样例：

```
5
at
touch
cheat
choose
tact
a
```

输出样例：

```
23
```

提示

连成的“龙”为 atoucheatactactouchoose。

预处理字符串，将所有拼接的可能转化可达数组 `g[N][N]==重复字符串长度`，然后对图进行遍历

```
#include <cstring>
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 21;

int n;
string word[N];
```

```
int g[N][N];
int used[N];
int ans;

void dfs(string dragon, int last)
{
    ans = max((int)dragon.size(), ans);
    used[last] ++ ;
    for (int i = 0; i < n; i ++ )
        if (g[last][i] && used[i] < 2)
            dfs(dragon + word[i].substr(g[last][i]), i);
    used[last] -- ;
}

int main()
{
    cin >> n;
    for (int i = 0; i < n; i ++ ) cin >> word[i];
    char start;
    cin >> start;

        // 预处理单词间重叠关系
    for (int i = 0; i < n; i ++ )
        for (int j = 0; j < n; j ++ )
        {
            string a = word[i], b = word[j];
            for (int k = 1; k < min(a.size(), b.size()); k ++ )
                if (a.substr(a.size() - k, k) == b.substr(0, k))
                {
                    g[i][j] = k;
                    break;
                }
        }

       // 从所有以start开头的单词开始搜索
    for (int i = 0; i < n; i ++ )
        if (word[i][0] == start)
            dfs(word[i], i);

    cout << ans << endl;

    return 0;
}


```

DFS之剪枝与优化

AcWing 165. 小猫爬山

翰翰和达达饲养了 N 只小猫，这天，小猫们要去爬山。

经历了千辛万苦，小猫们终于爬上了山顶，但是疲倦的它们再也不想徒步走下山了（呜咕>_<）。

翰翰和达达只好花钱让它们坐索道下山。

索道上的缆车最大承重量为 W，而 N 只小猫的重量分别是 C_1、$C_2 \ldots\ldots C_N$。

当然，每辆缆车上的小猫的重量之和不能超过 W。

每租用一辆缆车，翰翰和达达就要付 1 美元，所以他们想知道，最少需要付多少美元才能把这 N 只小猫都运送下山?

输入格式

第 1 行：包含两个用空格隔开的整数，N 和 W。

第 $2..N+1$ 行：每行一个整数，其中第 $i+1$ 行的整数表示第 i 只小猫的重量 C_i。

输出格式

输出一个整数，表示最少需要多少美元，也就是最少需要多少辆缆车。

数据范围

$1 \le N \le 18$,
$1 \le C_i \le W \le 10^8$

输入样例：

```
5 1996
1
2
1994
12
29
```

输出样例：

```
2
```

这题可以使用记忆化搜索或者使用深搜暴搜

深搜暴搜

- k >= res, return　　//剪枝，大于当前最优答案直接返回（后续更新不能使k减少）。
- 优先考虑决策少的组合（递归深度小）——将猫从大到小排序。

```
#include <iostream>
#include <algorithm>
using namespace std;

```

```cpp
const int N = 20;
int ans = N;
int cat[N];
int sum[N];
int n, m;

void dfs(int u, int k) {
    if (k >= ans) return; // 剪枝：当前缆车数已不小于已知最小值
    if (u == n) { // 所有小猫已分配
        ans = k;    //k一定是小于等于ans的
        return;
    }
    for (int i = 0; i < k; i++) { // 尝试放入已有缆车
        if (sum[i] + cat[u] <= m) {
            sum[i] += cat[u];
            dfs(u + 1, k);
            sum[i] -= cat[u]; // 回溯
        }
    }
    // 新开一辆缆车
    sum[k] = cat[u];
    dfs(u + 1, k + 1);
    sum[k] = 0; // 回溯
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        cin >> cat[i];
    }
    sort(cat, cat + n, greater<int>()); // 从大到小排序
   //  sort(cat,cat+n);
  //  reverse(cat,cat+n);
    dfs(0, 0); // 初始调用：处理第0只小猫，当前缆车数为0
    cout << ans << endl;
    return 0;
}
```

AcWing 166. 数独

数独 是一种传统益智游戏，你需要把一个 9×9 的数独补充完整，使得数独中每行、每列、每个 3×3 的九宫格内数字 $1 \sim 9$ 均恰好出现一次。

请编写一个程序填写数独。

输入格式

输入包含多组测试用例。

每个测试用例占一行，包含 81 个字符，代表数独的 81 个格内数据（顺序总体由上到下，同行由左到右）。

每个字符都是一个数字（$1-9$）或一个 `.` （表示尚未填充）。

您可以假设输入中的每个谜题都只有一个解决方案。

文件结尾处为包含单词 `end` 的单行，表示输入结束。

输出格式

每个测试用例，输出一行数据，代表填充完全后的数独。

输入样例：

```
4.....8.5.3.........7......2.....6.....8.4......1.......6.3.7.5..2.....1.4......
......52..8.4......3...9...5.1...6..2..7........3.....6...1..........7.4.......3.
end
```

输出样例：

```
417369825632158947958724316825437169791586432346912758289643571573291684164875293
416837529982465371735129468571298643293746185864351297647913852359682714128574936
```

.	2	7	3	8	.	.	1	.
.	1	.	.	.	6	7	3	5
.	.	.	.	.	.	.	2	9
3	.	5	6	9	2	.	8	.
.	.	.	.	.	.	.	.	.
.	6	.	1	7	3	5	.	3
6	4	.	.	.	.	.	.	.
9	5	1	8	.	.	.	7	.
.	8	.	.	6	5	3	4	.

要求:

1. 每一行1-9
2. 每一列1-9
3. 九宫格1-9

优化:

- 优化搜索顺序
  - 从当前能填合法数字最少的位置开始填数字
  - 使用ones存储数字i的二进制1的个数
  - 使用map存储可以当前二进制对应的数字·
- 排除等效冗余
  - 任意一个状态下,我们只需要找一个位置填数即可,而不是找所有的位置和可填的数字.

```
#include <cstring>
#include <iostream>

using namespace std;


const int N = 9, M = 1 << N;     // N=9（数独大小），M=512（2^9，所有可能的数字组合）

int ones[M];      // ones[i]：数字i的二进制1的个数
int map[M];      //map[i]：数字i对应的log2值，即可以选择的数字
int row[N], col[N], cell[3][3]; // 行、列、3×3宫的可用数字掩码
char str[100];
```

```cpp


void init()
{
    for(int i = 0; i < N; i ++)
        row[i] = col[i] = (1 << N) - 1;  //  初始所有数字可用（二进制全1）

    for(int i = 0; i < 3; i ++)
        for(int j = 0; j < 3; j ++)
            cell[i][j] = (1 << N) - 1;   // 3×3宫初始所有数字可用
}

inline int lowbit(int x)    // 返回 x 的二进制最低位的 1
{
    return x & (-x);
}

int get(int x, int y)       // 返回 (x,y) 可用的数字掩码
{
    return row[x] & col[y] & cell[x / 3][y / 3];
}

void draw(int x, int y, int t, bool is_set)     // x,y为当前位置，t为当前数字，is_set为是填入还是删除
{
    if(is_set)  str[x * N + y] = t + '1';
    else str[x * N + y] = '.';

    int v = 1 << t;
    if(!is_set) v = -v;

    row[x] -= v;
    col[y] -= v;
    cell[x / 3][y / 3] -= v;
}

bool dfs(int cnt)
{
    if(!cnt)    return true;        // 填入所有数字后，成功

    // 找可填数字最少的空格（优化搜索顺序）
    int x, y;
    int minv = 10;
    for(int i = 0; i < N; i ++)
        for(int j = 0; j < N; j ++)
            if(str[i * N + j] == '.')
            {
                int state = get(i,j);
                if(ones[state] < minv)
                {
                    minv = ones[state];
                    x = i, y = j;
                }
```

```
            }

    // 尝试所有可填数字
    int state = get(x, y);
    for(int i = state; i; i -= lowbit(i))
    {
        int t = map[lowbit(i)];      // 取出最低位的 1 对应的数字
        draw(x, y, t, true);
        if(dfs(cnt - 1)) return true;   // 若成功直接回溯到main函数
        draw(x, y, t, false);
    }
    return false;
}


int main()
{
    for(int i = 0; i< N; i ++)
       map[1 << i] = i;    // 将1 << i映射到i
    for(int i = 0; i < 1<< N; i ++)
        for(int j = 0; j < N; j ++)
            ones[i] += (i >> j) & 1;    // 计算i的二进制中1的个数

    while(cin >> str, str[0] != 'e')
    {
        // i是行号，j是列号，k是str索引
        init();
        int cnt = 0;    // 统计空格数量
        for(int i = 0, k = 0; i < N; i ++)
            for(int j = 0; j< N; j++, k++)
                if (str[k] != '.')
                  {
                    int t = str[k] - '1';
                    draw(i,j,t,true);
                  }
                else
                    cnt ++;
        dfs(cnt);
        puts(str);
    }
    return 0;
}

```